CNN Practical Implementation

Structured Notes

Transfer Learning Recap, Data Preparation Pipeline, Transformations, Dataset and DataLoader in PyTorch

Contents

1 Why Training a CNN from Scratch is Hard **2**
1.1 CNN Layer Hierarchy . . . 2
1.2 Parameter Scale Comparison . . . 2

2 Transfer Learning **2**
2.1 ImageNet Pre-trained Models . . . 3
2.2 Strategy 1: Feature Extraction . . . 3
2.3 Strategy 2: Fine-Tuning . . . 3
2.4 Freezing Layers in PyTorch . . . 3

3 Data Preparation for CNN **4**
3.1 Data Pipeline Requirements . . . 4
3.2 Folder Structure for Image Classification . . . 4
3.3 Label Encoding . . . 5

4 Image Transformations **5**
4.1 Types of Transformations . . . 6
4.2 Why Data Augmentation is Needed . . . 6
4.3 Training vs Validation/Test Transforms . . . 6
4.4 Transform Pipeline in PyTorch . . . 6
4.5 ImageNet Normalization Values . . . 7

5 Pre-processing Best Practices **7**
5.1 OpenCV for Pre-processing . . . 7
5.2 Pre-processing with OpenCV vs PyTorch Transforms . . . 8

6 Dataset and DataLoader in PyTorch **8**
6.1 Required Libraries . . . 8
6.2 Dataset vs DataLoader . . . 8
6.3 ImageFolder . . . 8
6.4 Batch Shape Interpretation . . . 8
6.5 Directory Setup for Training . . . 9

7 Real-World Data Complications **9**

8 Summary **9**

1 Why Training a CNN from Scratch is Hard

Building a CNN from scratch is computationally very heavy. Typically, there is not enough data available to train a CNN effectively, and even when sufficient data exists, the computational resources required are prohibitive.

Core Problems with Training from Scratch

- **Many parameters** — Modern CNNs contain millions of trainable parameters
- **Large dataset requirement** — A model with many parameters needs enough data to learn stable patterns
- **High computation** — GPU memory, training time, and tuning overhead
- **Long training time** — Training on millions of images is time-consuming
- **Careful tuning** — Learning rate, regularization, debugging unstable training

1.1 CNN Layer Hierarchy

As depth increases, the network becomes more powerful but also more expensive to train:

- **Early layers** learn general features: edges, corners, and textures
- **Middle layers** learn combinations: shapes, repeated patterns, object parts
- **Deep layers** learn class-specific structures and semantic patterns

1.2 Parameter Scale Comparison

Even though CNNs are more parameter-efficient than fully connected ANNs for images, modern CNNs still contain millions of trainable parameters.

Model	Parameters
Small custom CNN	Thousands to millions
ResNet-18	∼11.6 million

Data Requirement Problem

A model with many parameters needs enough data to learn stable patterns. If the dataset is too small, the model may memorize training images — this is called **overfitting**. The real question is not "can we train the model?" but "can we train it well enough, fast enough, and with enough data?"

ResNet-18 is trained on 1,000 classes. There are different versions of ResNet: ResNet-18, ResNet-51, ResNet-101.

2 Transfer Learning

Transfer Learning

Transfer learning is the practice of reusing knowledge learned from one task to solve another related task. Instead of training a CNN from scratch, a pre-trained model's learned features are leveraged for a new, related task.

Every image classification task is fundamentally similar: the model must detect edges, patterns, and complex features to relate those patterns. Unless the task is very domain-specific, normal image classification tasks are almost the same.

2.1 ImageNet Pre-trained Models

A CNN model trained on ImageNet already knows how to detect edges, textures, corners, color gradients, shapes, object parts, and many more visual features.

ImageNet Dataset:

- **Total images:** 1.2 million
- **Object classes:** 1,000
- **Images per class:** ∼1,200

ImageNet remains the biggest open-source dataset available to train image models. It is still the standard benchmark for pre-trained models.

Because of the hierarchical nature of CNN feature learning, transfer learning is very useful. Even if the new task is different, early visual features (edges, gradients, textures) are still valuable.

2.2 Strategy 1: Feature Extraction

In feature extraction, **all layers are frozen** except the last classification layer. The dense layers (fully connected layers) after the pooling operations are replaced to match the new task's requirements.

- Freeze all convolutional layers
- Replace the final classification layer with new dense layers
- Only train the new classification head
- More simple and effective for small datasets

2.3 Strategy 2: Fine-Tuning

When the data is task-specific and may not align well with a pre-trained model trained on ImageNet, fine-tuning is required. Instead of freezing all layers, some deeper (task-specific) layers are allowed to update their weights during training.

Fine-Tuning

In fine-tuning, instead of freezing all layers, some deeper layers are allowed to update during training. This is useful when the new task's data distribution differs significantly from the pre-training data (ImageNet).

2.4 Freezing Layers in PyTorch

Layers are frozen by turning off the gradients. In PyTorch, this is done by setting `param.requires_grad = False`.

Freezing Layers in PyTorch

```
import torchvision.models as models

# Load pre-trained ResNet-18
model = models.resnet18(pretrained=True)

# Freeze all parameters
for param in model.parameters():
    param.requires_grad = False

# Replace final classification layer
# Original: fc layer with 512 input features, 1000 output classes
model.fc = nn.Linear(512, num_classes)
```

Listing 1: Freezing pre-trained model parameters

When layers are frozen:

- Gradients are not computed for those layers
- Parameters remain fixed during training
- Training becomes faster because fewer parameters must be optimized

ResNet-18 Architecture Detail

The final layer before modification has 512 output features. Since ResNet-18 is trained on 1,000 ImageNet classes, the original output layer maps 512 features to 1,000 classes. When all trainable parameters are frozen via `requires_grad = False`, the trainable parameter count drops to zero. Only the newly added classification layer is trainable.

3 Data Preparation for CNN

The data preparation pipeline is the most important part of any ML/DL application. The initial and first most important pipeline is the dataset pre-processing.

Why Data Preparation Matters

A CNN does not read images the way humans do. For a CNN, an image must be converted into a structured tensor with:

- **Fixed size** — consistent height and width
- **Numeric values** — pixel values as numbers
- **Correct channel order** — RGB channels properly arranged
- **Consistent normalization** — standardized value ranges

If the data pipeline is incorrect:

- Tensor shapes may break
- Images may appear distorted
- Model performance may degrade
- Training may become unstable

Critical Insight

Students often rush to model training before understanding the data. A wrong data pipeline can **silently destroy** model performance. Data preparation is not just loading files — it is the central part of deep learning practice. Before trusting a model, first trust the data pipeline.

3.1 Data Pipeline Requirements

Before model training, the following must be ensured:

1. Images are loaded correctly
2. Labels are assigned and collected properly
3. Transformations are applied correctly
4. Batches are formed correctly

3.2 Folder Structure for Image Classification

For a classification task (e.g., cats vs dogs), the data should be organized into separate directories:

Standard Directory Structure:

```
data/
    train/
        cat/
            img001.jpg
            img002.jpg
        dog/
            img001.jpg
            img002.jpg
    val/
        cat/
        dog/
    test/
        cat/
        dog/
```

The split should not be randomly assigned at runtime. The folders should be fixed after performing the initial train-test-validation split.

The recommended workflow:

1. Download the raw data
2. Use Pandas to perform the train-test-validation split
3. Create a data frame and organize into the folder structure shown above

3.3 Label Encoding

Models do not understand string labels like "cat" or "dog." Labels must be converted to numeric form.

One-Hot Encoding

One-hot encoding converts every label to a numeric form. After the logits are converted to probabilities, the encoding maps back to the original string labels. One-hot encoding is the most commonly used method for label encoding in classification tasks.

The `class_to_index` mapping should always be inspected before training to verify that indexes are appropriately mapped to the correct labels.

4 Image Transformations

Real images do not all have the same height, width, orientation, brightness, and composition. The same environment cannot be replicated exactly when capturing images — there will always be differences in exposure, brightness, composition, and other factors.

4.1 Types of Transformations

Transformation	Description
Resize	Ensures all images have a fixed, consistent size
Random Horizontal Flip	Mirror image horizontally (with probability p)
Random Vertical Flip	Mirror image vertically
Random Rotation	Rotate the image by a small angle
Random Crop	Crop a random portion of the image
To Tensor	Convert pixel values to PyTorch tensors
Normalize	Standardize values using mean and standard deviation

4.2 Why Data Augmentation is Needed

Data augmentation is applied to mimic real-world scenarios. If the dataset is too clean and well-presented, it does not actually represent the real world. When the model classifies real-world data, it encounters various orientations, lighting conditions, and other variations. Augmentation prepares the model for these real-world possibilities.

4.3 Training vs Validation/Test Transforms

Critical Distinction

Data augmentation (flips, rotations, brightness adjustments) should be applied **only to training data**. Validation and test data should **not** receive augmentation transforms. Applying augmentation to validation/test data causes **data leakage** and prevents accurate evaluation of the model's actual performance.

Step	Training	Validation / Test
Resize	✓	✓
Augmentation (flip, rotate, etc.)	✓	×
To Tensor	✓	✓
Normalize	✓	✓

4.4 Transform Pipeline in PyTorch

Training and Validation Transforms

```
from torchvision import transforms

# ImageNet normalization values
imagenet_mean = [0.485, 0.456, 0.406]
imagenet_std = [0.229, 0.224, 0.225]

# Training transforms: resize + augmentation + tensor + normalize
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=imagenet_mean, std=imagenet_std)
])

```

```
# Validation/Test transforms: resize + tensor + normalize (NO augmentation)
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=imagenet_mean, std=imagenet_std)
])
```

Listing 2: Defining transform pipelines

Random Horizontal Flip Probability

With `p=0.5`, approximately 50% of images in each batch will be flipped horizontally. Not every image gets flipped — this randomness mimics real-world variation. Additional augmentations like vertical flip, rotation, and crop can also be applied, but are not always necessary.

4.5 ImageNet Normalization Values

Pre-trained ResNet models typically use ImageNet normalization values because ResNet was originally trained on the ImageNet dataset. Keeping the same normalization values ensures consistency with the learned feature representations.

These standard values (mean and standard deviation per RGB channel) are computed from the entire ImageNet dataset.

5 Pre-processing Best Practices

5.1 OpenCV for Pre-processing

While PyTorch's `torchvision.transforms` module can handle image pre-processing, it is recommended to perform all pre-processing steps with **OpenCV** instead. OpenCV is specifically designed for image operations and is faster than PyTorch's built-in transforms for this purpose.

The recommended pipeline:

1. Load and pre-process images using OpenCV (`cv2.resize`, etc.)
2. Convert pre-processed images to tensors
3. Save as `.pt` files (PyTorch format) or `.pkl` files (pickle format)
4. Load the `.pt` files directly during training

File Formats for Processed Data

- **.pt files** — Best for PyTorch workflows. Contains tensor data in PyTorch's native format.
- **.pkl files** — Pickle format, works better with TensorFlow.

A `.pt` file contains all the information of the image in tensor form. This eliminates the need to share the complete raw dataset — only the training, testing, and validation `.pt` files need to be shared.

5.2 Pre-processing with OpenCV vs PyTorch Transforms

Aspect	OpenCV	PyTorch Transforms
Speed	Faster, purpose-built for images	Slower for image operations
Flexibility	Full control over operations	Higher-level abstraction
Use case	Production pipelines	Quick prototyping
Recommendation	Preferred for all pre-processing	Acceptable but not optimal

6 Dataset and DataLoader in PyTorch

6.1 Required Libraries

Import Statements

```python
import os
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

Listing 3: Libraries for CNN data pipeline

`torchvision.datasets` can import standard datasets, but custom datasets from sources like Kaggle can also be used.

6.2 Dataset vs DataLoader

Dataset and DataLoader

- **Dataset** returns only a single image, one at a time. It defines how to access individual data samples.
- **DataLoader** handles batching, shuffling, and iteration. It groups samples into batches and manages the data loading process during training.

6.3 ImageFolder

`ImageFolder` loads images from a directory structure and automatically assigns labels based on folder names. While not always used in practice (custom datasets from Kaggle may require different loading approaches), it is important to understand.

6.4 Batch Shape Interpretation

A batch shape of `[16, 3, 224, 224]` means:

Dimension	Value	Meaning
0	16	Batch size (number of images)
1	3	Channels (RGB)
2	224	Height (pixels)
3	224	Width (pixels)

6.5 Directory Setup for Training

Directory Structure Setup

```
import os

data_dir = "path/to/data"
train_dir = os.path.join(data_dir, "train")
val_dir = os.path.join(data_dir, "val")
test_dir = os.path.join(data_dir, "test")
```

Listing 4: Setting up data directories

After splitting the data, the splits can be verified to confirm the distribution is appropriate before proceeding with training.

7 Real-World Data Complications

When working with real-world data, additional complications arise that are not present in curated datasets like ImageNet.

Class Imbalance

Class imbalance occurs when the distribution of samples across classes is unequal. For example, ImageNet has approximately 1,200 images per class, but real-world data collection often yields biased distributions — one class may have significantly more samples than another (e.g., 70 dog images vs 30 cat images out of 100 total).

Additional real-world data challenges include:

- **Corrupt data** — damaged or unreadable image files
- **Noisy labels** — incorrectly labeled samples
- **Metadata issues** — missing or inconsistent metadata

Stratified sampling is one approach to address class imbalance during the train-test-validation split.

8 Summary

- Training a CNN from scratch requires millions of parameters, large datasets, and extensive computation
- **Transfer learning** reuses knowledge from pre-trained models (typically trained on ImageNet with 1.2 million images across 1,000 classes)
- Two transfer learning strategies: **feature extraction** (freeze all, train last layer) and **fine-tuning** (allow deeper layers to update)
- Layers are frozen in PyTorch using `param.requires_grad = False`
- **Data preparation** is the most critical step — a wrong pipeline silently destroys model performance
- Images must be converted to structured tensors with fixed size, numeric values, correct channel order, and consistent normalization
- **Augmentation** (flips, rotations) is applied only to training data, never to validation/test data, to prevent data leakage
- Pre-trained models use **ImageNet normalization values** for consistency
- **OpenCV** is recommended over PyTorch transforms for pre-processing operations
- Processed data can be saved as `.pt` files for efficient loading during training
- **Dataset** returns single samples; **DataLoader** handles batching, shuffling, and iteration
- Real-world data challenges include class imbalance, corrupt data, and noisy labels